

Telco Customer Churn Prediction

Name: Ali Baba

Id:CSC-23S-093

Course: Machine Learning

Assignment: Customer Churn Prediction

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:

```
df = pd.read_csv('Telco Customer Churn.csv')
print(df)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	\
0	7590-VHVEG	Female	0	Yes	No	1	
1	5575-GNVDE	Male	0	No	No	34	
2	3668-QPYBK	Male	0	No	No	2	
3	7795-CF0CW	Male	0	No	No	45	
4	9237-HQITU	Female	0	No	No	2	
...	...	...	...	...	...	...	
7038	6840-RESVB	Male	0	Yes	Yes	24	
7039	2234-XADUH	Female	0	Yes	Yes	72	
7040	4801-JZAZL	Female	0	Yes	Yes	11	
7041	8361-LTMKD	Male	1	Yes	No	4	
7042	3186-AJIEK	Male	0	No	No	66	

	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	\
0	No	No phone service	DSL	No	...	
1	Yes	No	DSL	Yes	...	
2	Yes	No	DSL	Yes	...	
3	No	No phone service	DSL	Yes	...	
4	Yes	No	Fiber optic	No	...	
...	...	...	...	...	...	
7038	Yes	Yes	DSL	Yes	...	
7039	Yes	Yes	Fiber optic	No	...	
7040	No	No phone service	DSL	Yes	...	
7041	Yes	Yes	Fiber optic	No	...	
7042	Yes	No	Fiber optic	Yes	...	

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	No	No	No	No	Month-to-month	
1	Yes	No	No	No	One year	

2	No	No	No	No	Month-to-month
3	Yes	Yes	No	No	One year
4	No	No	No	No	Month-to-month
...	...	...	...	...	...
7038	Yes	Yes	Yes	Yes	One year
7039	Yes	No	Yes	Yes	One year
7040	No	No	No	No	Month-to-month
7041	No	No	No	No	Month-to-month
7042	Yes	Yes	Yes	Yes	Two year

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	\
0	Yes	Electronic check	29.85	29.85	
1	No	Mailed check	56.95	1889.5	
2	Yes	Mailed check	53.85	108.15	
3	No	Bank transfer (automatic)	42.30	1840.75	
4	Yes	Electronic check	70.70	151.65	
...	...	...	...	...	
7038	Yes	Mailed check	84.80	1990.5	
7039	Yes	Credit card (automatic)	103.20	7362.9	
7040	Yes	Electronic check	29.60	346.45	
7041	Yes	Mailed check	74.40	306.6	
7042	Yes	Bank transfer (automatic)	105.65	6844.5	

Churn	
0	No
1	No
2	Yes
3	No
4	Yes
...	...
7038	No
7039	No
7040	No
7041	Yes
7042	No

[7043 rows x 21 columns]

In []:

In [229]:

`df.head()`

Out[229]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Intern
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	
4	9237-HQITU	Female	0	No	No	2	Yes	No	I

5 rows × 21 columns

In [230]:

```
df.shape
```

Out[230]:

```
(7043, 21)
```

In [231]:

```
df.columns
```

Out[231]:

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
      'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
      'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

In [232]:

```
df.duplicated().sum()
```

Out[232]:

```
np.int64(0)
```

In [233]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7043 entries, 0 to 7042
```

```
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
0	customerID	7043 non-null	object
1	gender	7043 non-null	object
2	SeniorCitizen	7043 non-null	int64
3	Partner	7043 non-null	object
4	Dependents	7043 non-null	object
5	tenure	7043 non-null	int64
6	PhoneService	7043 non-null	object
7	MultipleLines	7043 non-null	object
8	InternetService	7043 non-null	object
9	OnlineSecurity	7043 non-null	object
10	OnlineBackup	7043 non-null	object
11	DeviceProtection	7043 non-null	object
12	TechSupport	7043 non-null	object
13	StreamingTV	7043 non-null	object
14	StreamingMovies	7043 non-null	object
15	Contract	7043 non-null	object
16	PaperlessBilling	7043 non-null	object
17	PaymentMethod	7043 non-null	object
18	MonthlyCharges	7043 non-null	float64
19	TotalCharges	7043 non-null	object
20	Churn	7043 non-null	object

```
dtypes: float64(1), int64(2), object(18)
```

```
memory usage: 1.1+ MB
```

In [234]:

```
df.isnull().sum()
```

```
Out[234]:
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines    0
InternetService 0
OnlineSecurity  0
OnlineBackup     0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64
```

```
In [235]:
```

```
df["TotalCharges"].dtype
```

```
Out[235]:
```

```
dtype('O')
```

```
In [236]:
```

```
df["TotalCharges"].head(20)
```

```
Out[236]:
```

```
0      29.85
1    1889.5
2     108.15
3   1840.75
4     151.65
5     820.5
6    1949.4
7     301.9
8    3046.05
9    3487.95
10     587.45
11     326.8
12    5681.1
13    5036.3
14    2686.05
15    7895.15
16    1022.95
17    7382.25
18     528.35
19    1862.9
```

```
Name: TotalCharges, dtype: object
```

```
In [237]:
```

```
df["TotalCharges"].value_counts().head()
```

```
Out[237]:
```

```
TotalCharges
      11
20.2    11
19.75    9
20.05    8
19.9     8
Name: count, dtype: int64
```

Dataset Inspection

The Telco Customer Churn dataset contains 7,043 customer records and 21 columns. Each row represents an individual customer while each column contains information about the customers demographics subscribed services account details and billing information. The `Churn` column is the target variable that indicates whether a customer left the company.

In [238]:

```
df.head()
df.shape
df.columns
df.info()
df.isnull().sum()
df.duplicated().sum()
df.describe()
df["TotalCharges"].dtype
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   object
7   MultipleLines           7043 non-null   object
8   InternetService        7043 non-null   object
9   OnlineSecurity          7043 non-null   object
10  OnlineBackup            7043 non-null   object
11  DeviceProtection        7043 non-null   object
12  TechSupport             7043 non-null   object
13  StreamingTV             7043 non-null   object
14  StreamingMovies         7043 non-null   object
15  Contract                7043 non-null   object
16  PaperlessBilling        7043 non-null   object
17  PaymentMethod           7043 non-null   object
18  MonthlyCharges          7043 non-null   float64
19  TotalCharges            7043 non-null   object
20  Churn                   7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Out[238]:
dtype('O')

In [239]:

```
df["TotalCharges"].value_counts().tail()
```

```
Out[239]:
TotalCharges
6849.4      1
692.35     1
130.15     1
3211.9     1
6844.5     1
Name: count, dtype: int64
```

```
In [240]:
```

```
df["TotalCharges"].str.strip().eq("").sum()
```

```
Out[240]:
np.int64(11)
```

```
In [241]:
```

```
df["TotalCharges"] = df["TotalCharges"].replace(" ", np.nan)
```

```
In [242]:
```

```
df["TotalCharges"].isnull().sum()
```

```
Out[242]:
np.int64(11)
```

```
In [243]:
```

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"])
```

```
In [244]:
```

```
df["TotalCharges"].dtype
```

```
Out[244]:
dtype('float64')
```

```
In [245]:
```

```
df["TotalCharges"] = df["TotalCharges"].fillna(
    df["TotalCharges"].median()
)
```

```
In [246]:
```

```
df["TotalCharges"].isnull().sum()
```

```
Out[246]:
np.int64(0)
```

```
In [247]:
```

```
df["TotalCharges"].dtype
```

```
Out[247]:
dtype('float64')
```

Data Cleaning

The `TotalCharges` column was initially stored as an object because some records contained blank spaces. There were 11 blank values in this column. The blank values were converted to missing values and `TotalCharges` was then converted to a numeric data type. The remaining missing values were replaced using the median to avoid losing customer records.

In [248]:

```
df.isnull().sum()
```

Out[248]:

```
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64
```

In [249]:

```
df["customerID"].nunique()
```

Out[249]:

```
7043
```

In [250]:

```
df["Churn"].value_counts()
```

Out[250]:

```
Churn
No      5174
Yes     1869
Name: count, dtype: int64
```

In [251]:

```
df["Churn"].value_counts(normalize=True) * 100
```

Out[251]:

```
Churn
No      73.463013
Yes     26.536987
Name: proportion, dtype: float64
```

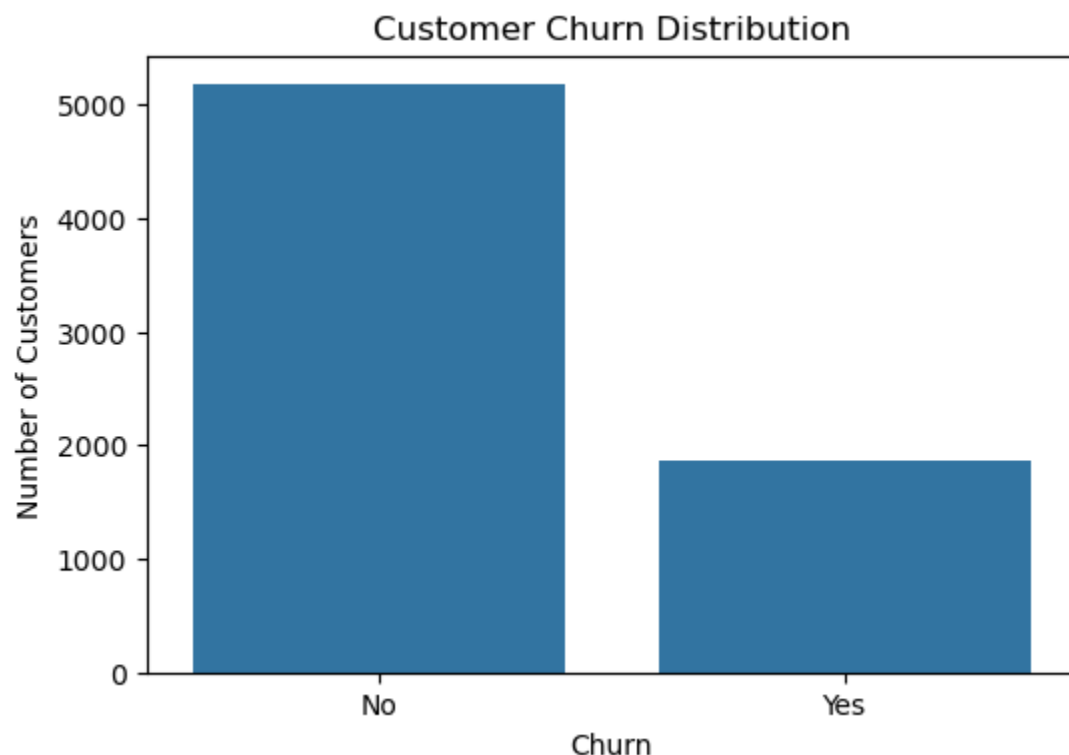
In [252]:

```
plt.figure(figsize=(6,4))

sns.countplot(data=df, x="Churn")

plt.title("Customer Churn Distribution")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")
```

```
plt.show()
```



Churn Distribution

The target variable `Churn` contains two classes: Yes and No. The number of customers who did not churn is considerably higher than the number of customers who churned. This indicates that the dataset is imbalanced with the churn class representing the minority class.

```
In [253]:
```

```
numeric_columns = df.select_dtypes(include=np.number).columns
```

```
numeric_columns
```

```
Out[253]:
```

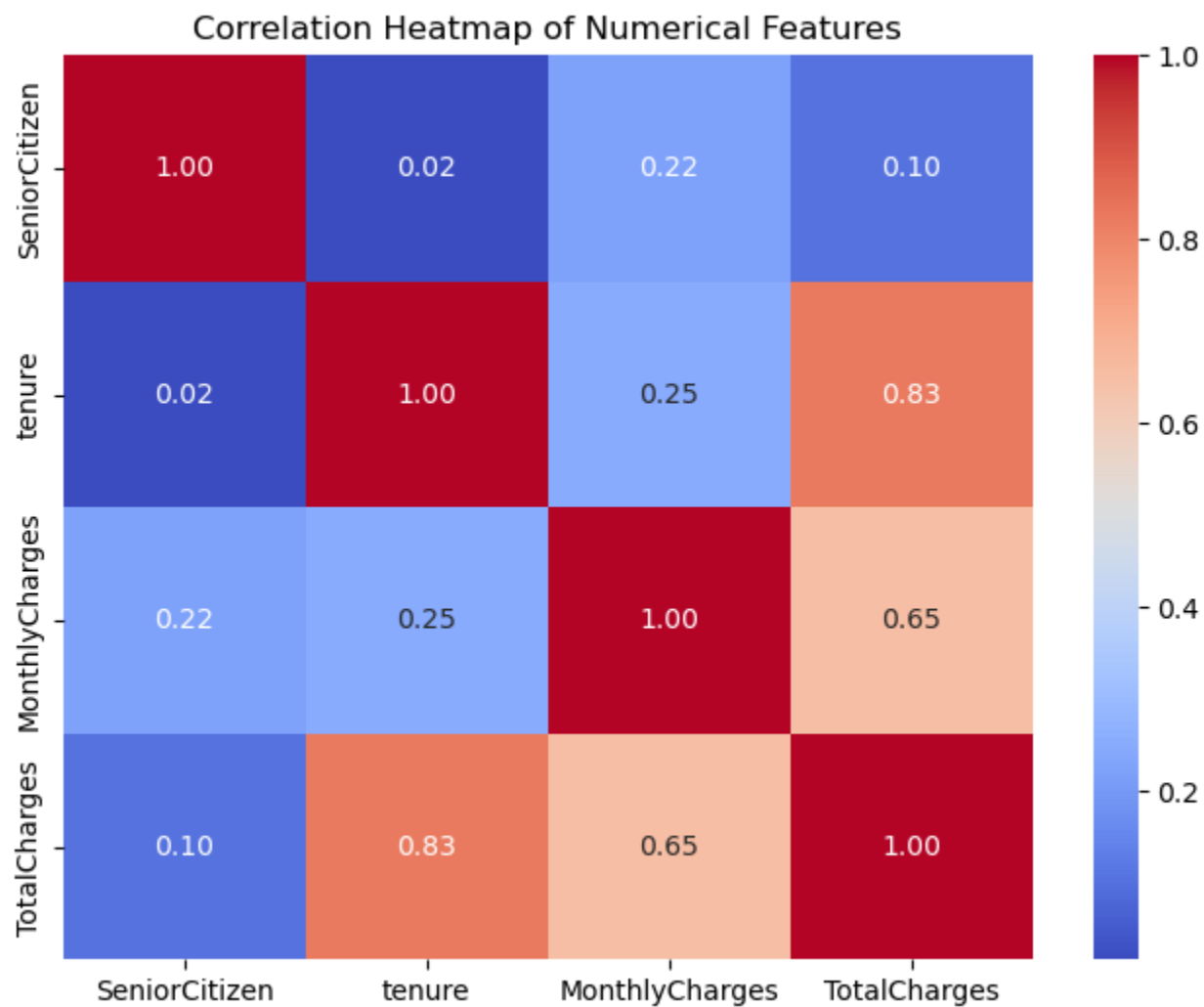
```
Index(['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges'], dtype='object')
```

```
In [254]:
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(  
    df[numeric_columns].corr(),  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"  
)
```

```
plt.title("Correlation Heatmap of Numerical Features")  
plt.show()
```



In [255]:

```
contract_churn = pd.crosstab(
    df["Contract"],
    df["Churn"],
    normalize="index"
) * 100
```

contract_churn

Out[255]:

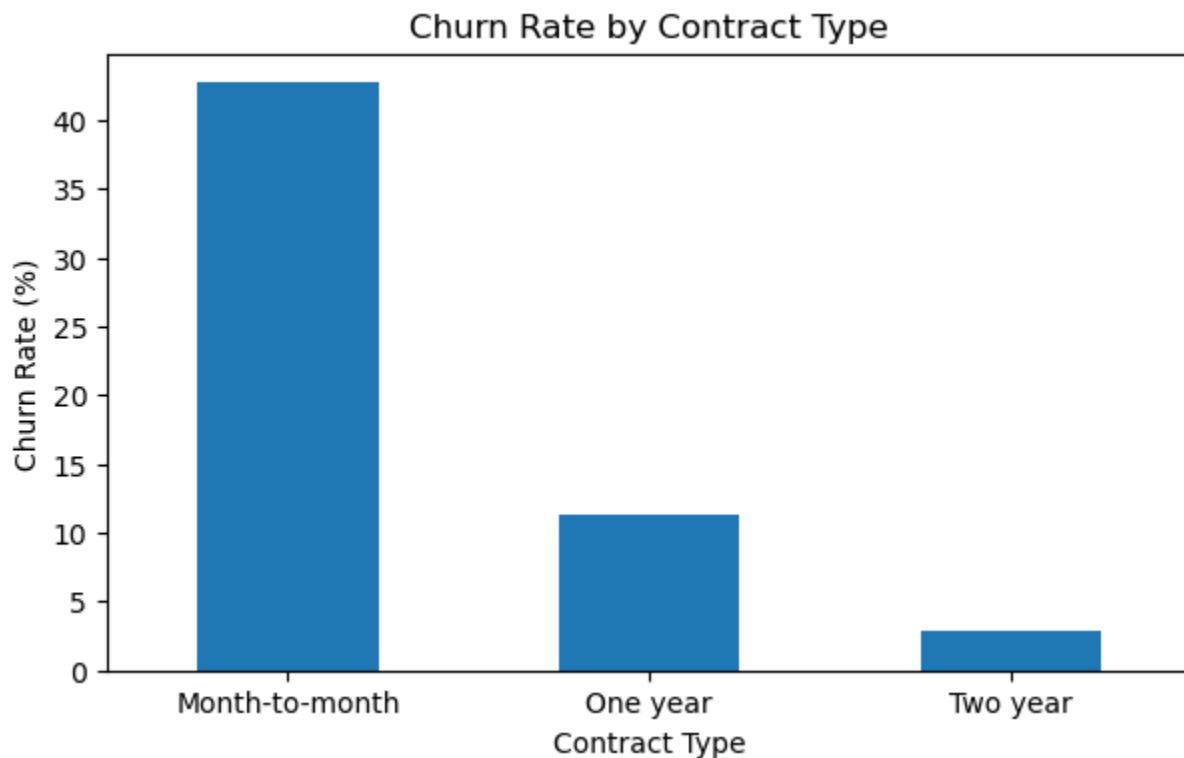
	Churn	No	Yes
Contract			
Month-to-month	57.290323	42.709677	
One year	88.730482	11.269518	
Two year	97.168142	2.831858	

In [256]:

```
contract_churn["Yes"].plot(kind="bar", figsize=(7,4))

plt.title("Churn Rate by Contract Type")
plt.xlabel("Contract Type")
plt.ylabel("Churn Rate (%)")
```

```
plt.xticks(rotation=0)
plt.show()
```



Interpretation: Customers with month to month contracts have the highest churn rate (42.71%) while customers with two year contracts have the lowest churn rate (2.83%). This suggests that customers with longer term contracts are more likely to stay with the company.

In [257]:

```
internet_churn = pd.crosstab(
    df["InternetService"],
    df["Churn"],
    normalize="index"
) * 100

internet_churn
```

Out[257]:

	Churn	No	Yes
InternetService			
DSL	81.040892	18.959108	
Fiber optic	58.107235	41.892765	
No	92.595020	7.404980	

Interpretation:

Fiber optic customers have the highest churn rate (41.89%) while customers with no internet service have the lowest churn rate (7.40%). This indicates that InternetService type may be an important factor associated with customer churn.

In [258]:

```
df["AvgMonthlyCharge"] = df["TotalCharges"] / df["tenure"]
```

In [259]:

```
df["AvgMonthlyCharge"] = np.where(
    df["tenure"] > 0,
    df["TotalCharges"] / df["tenure"],
    df["MonthlyCharges"]
)
```

In [260]:

```
df[["tenure", "TotalCharges", "MonthlyCharges", "AvgMonthlyCharge"]].head()
```

Out[260]:

	tenure	TotalCharges	MonthlyCharges	AvgMonthlyCharge
0	1	29.85	29.85	29.850000
1	34	1889.50	56.95	55.573529
2	2	108.15	53.85	54.075000
3	45	1840.75	42.30	40.905556
4	2	151.65	70.70	75.825000

In [261]:

```
df["IsNewCustomer"] = (df["tenure"] < 6).astype(int)
```

In [262]:

```
df[["tenure", "IsNewCustomer"]].head(10)
```

Out[262]:

	tenure	IsNewCustomer
0	1	1
1	34	0
2	2	1
3	45	0
4	2	1
5	8	0
6	22	0
7	10	0
8	28	0
9	62	0

In [263]:

```
df[["tenure", "IsNewCustomer"]].head(10).to_string(index=False)
```

```
Out[263]:
```

'tenure	IsNewCustomer	1	1	34	0	2
1	45	0	2	1	8	0
0	10	0	28	0	62	0

```
In [264]:
```

```
service_cols = [
    "PhoneService",
    "MultipleLines",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies"
]
```

```
In [265]:
```

```
service_cols
```

```
Out[265]:
```

```
['PhoneService',
 'MultipleLines',
 'OnlineSecurity',
 'OnlineBackup',
 'DeviceProtection',
 'TechSupport',
 'StreamingTV',
 'StreamingMovies']
```

```
In [266]:
```

```
df["ServiceCount"] = df[service_cols].eq("Yes").sum(axis=1)
```

```
In [267]:
```

```
df[["ServiceCount"] + service_cols].head()
```

```
Out[267]:
```

	ServiceCount	PhoneService	MultipleLines	OnlineSecurity	OnlineBackup	DeviceProtection	TechSup
0	1	No	No phone service	No	Yes	No	
1	3	Yes	No	Yes	No	Yes	
2	3	Yes	No	Yes	Yes	No	
3	3	No	No phone service	Yes	No	Yes	
4	1	Yes	No	No	No	No	

Feature Engineering Justification

AvgMonthlyCharge: Helps identify customers whose average monthly spending may be associated with churn. IsNewCustomer: Identifies customers who have been with the company for less than 6 months as new customers may have different churn behavior. ServiceCount: Represents the number of services used by a customer and may help understand whether service engagement is related to churn.

In [268]:

```
df = df.drop("customerID", axis=1)
```

In [269]:

```
df.shape
```

Out[269]:

(7043, 23)

In [270]:

```
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})
```

In [271]:

```
df["Churn"].value_counts()
```

Out[271]:

Churn

0 5174

1 1869

Name: count, dtype: int64

In [272]:

```
X = df.drop("Churn", axis=1)
y = df["Churn"]
```

In [273]:

```
print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (7043, 22)

y shape: (7043,)

In [274]:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

In [275]:

```
print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

X_train: (5634, 22)

X_test: (1409, 22)

y_train: (5634,)

y_test: (1409,)

In [276]:

```
numeric_features = X_train.select_dtypes(include=np.number).columns.tolist()
categorical_features = X_train.select_dtypes(include="object").columns.tolist()

print("Numerical Features:")
print(numeric_features)
```

```
print("\nCategorical Features:")
print(categorical_features)
```

Numerical Features:

```
['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges', 'AvgMonthlyCharge', 'IsNewCustomer', 'ServiceCount']
```

Categorical Features:

```
['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod']
```

In [277]:

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
```

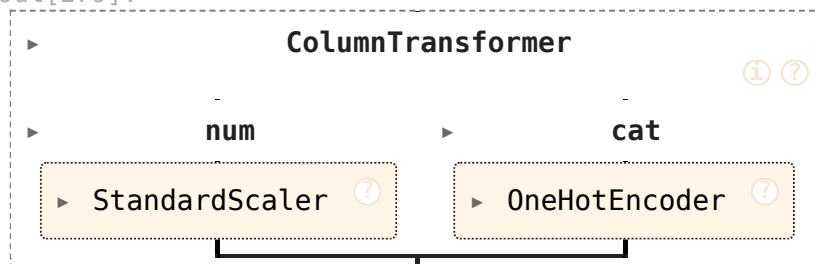
In [278]:

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)
```

In [279]:

```
preprocessor
```

Out[279]:



In [280]:

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
```

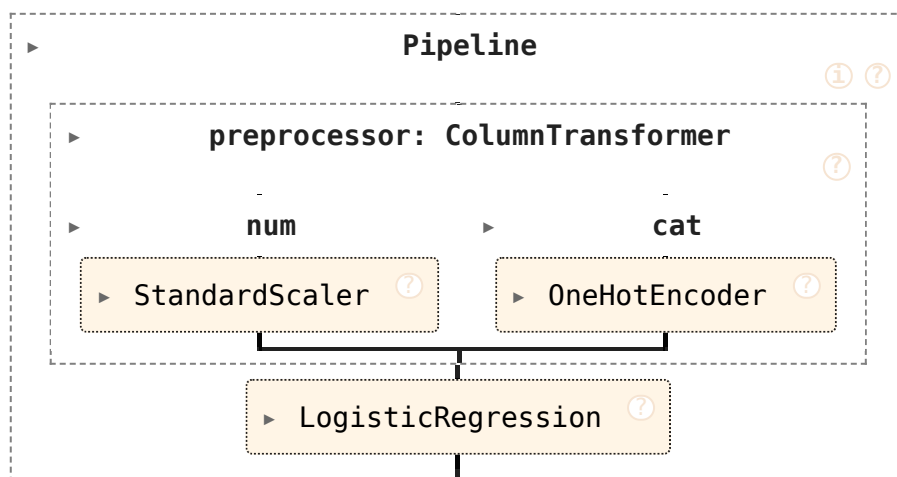
In [281]:

```
lr_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(
        class_weight="balanced",
        random_state=42,
        max_iter=1000
    ))
])
```

In [282]:

```
lr_model
```

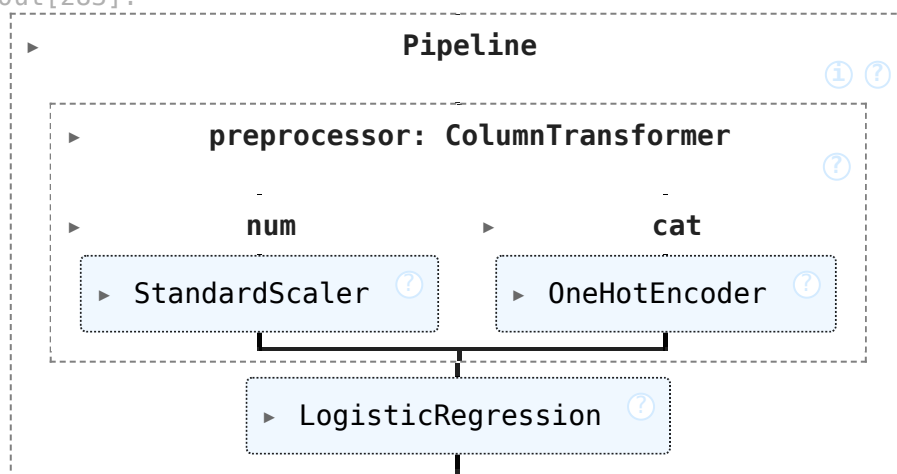
Out[282]:



In [283]:

```
lr_model.fit(X_train, y_train)
```

Out[283]:



In [284]:

```
y_pred_lr = lr_model.predict(X_test)
```

In [285]:

```
y_prob_lr = lr_model.predict_proba(X_test)[:, 1]
```

In [286]:

```
print(y_pred_lr[:10])
print(y_prob_lr[:10])
```

```
[0 1 0 1 0 1 1 0 0 1]
```

```
[0.11133481 0.83066894 0.15526037 0.60812164 0.07447748 0.78642512
 0.67494289 0.25646748 0.01418904 0.56750687]
```

In [287]:

```
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score
```

In [288]:

```
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)
roc_auc_lr = roc_auc_score(y_test, y_prob_lr)

print("Precision:", precision_lr)
```

```
print("Recall:", recall_lr)
print("F1 Score:", f1_lr)
print("ROC-AUC:", roc_auc_lr)
```

Precision: 0.5067796610169492
Recall: 0.7994652406417112
F1 Score: 0.6203319502074689
ROC-AUC: 0.8471879924565346

Logistic Regression Results Logistic Regression was trained using standardized numerical features and one hot encoded categorical features. Class balancing was applied to give more importance to the minority churn class. The model achieved a precision of 0.507 recall of 0.799 F1 score of 0.620 and ROC AUC of 0.847. The high recall indicates that the model was able to identify most customers who were likely to churn.

Decision Tree

In [289]:

```
from sklearn.tree import DecisionTreeClassifier
```

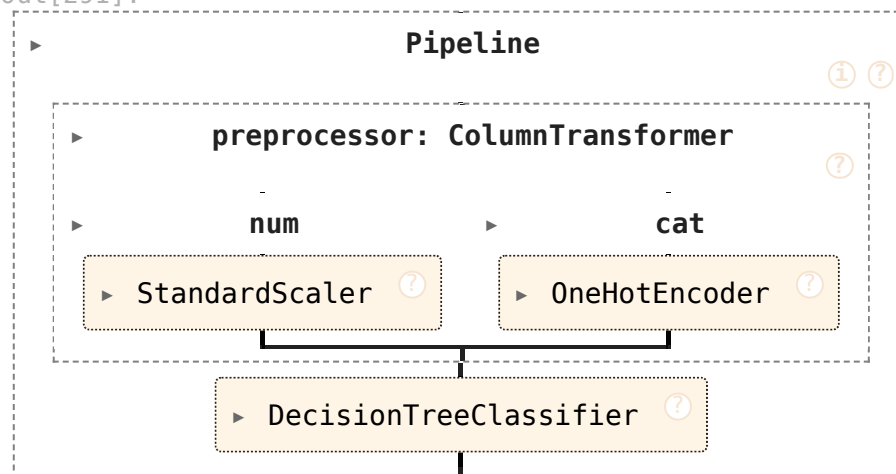
In [290]:

```
dt_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", DecisionTreeClassifier(
        class_weight="balanced",
        random_state=42
    ))
])
```

In [291]:

```
dt_model
```

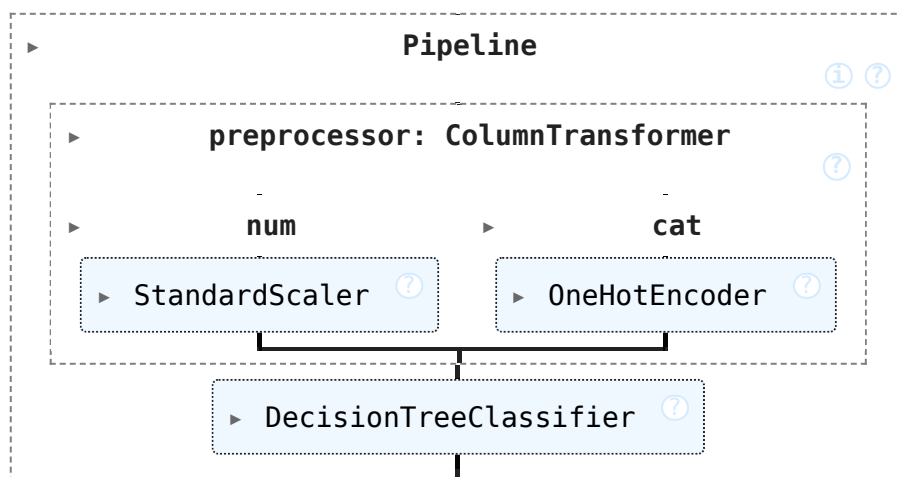
Out[291]:



In [292]:

```
dt_model.fit(X_train, y_train)
```

Out[292]:



In [293]:

```
y_pred_dt = dt_model.predict(X_test)
```

In [294]:

```
y_prob_dt = dt_model.predict_proba(X_test)[: , 1]
```

In [295]:

```
print(y_pred_dt[:10])
print(y_prob_dt[:10])
```

```
[0 0 0 0 0 0 0 0 0 1]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
```

In [296]:

```
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)
roc_auc_dt = roc_auc_score(y_test, y_prob_dt)
```

```
print("Precision:", precision_dt)
print("Recall:", recall_dt)
print("F1 Score:", f1_dt)
print("ROC-AUC:", roc_auc_dt)
```

```
Precision: 0.4946808510638298
Recall: 0.49732620320855614
F1 Score: 0.496
ROC-AUC: 0.6566250742721331
```

Decision Tree Results: The Decision Tree achieved a precision of 0.495 recall of 0.497 F1 score of 0.496 and ROC AUC of 0.657. Its performance was lower than Logistic Regression particularly in recall and ROC AUC.

Random Forest

In [297]:

```
from sklearn.ensemble import RandomForestClassifier
```

In [298]:

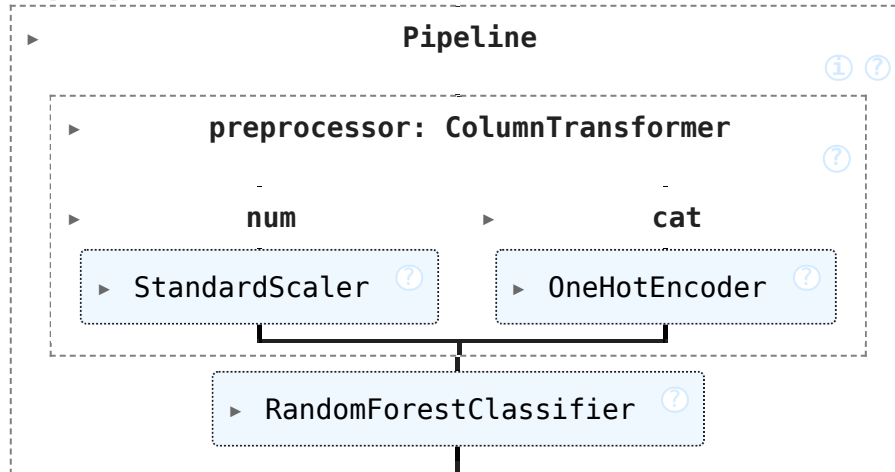
```
rf_model = Pipeline([
    ("preprocessor", preprocessor),
```

```
("classifier", RandomForestClassifier(
    class_weight="balanced",
    random_state=42
))
])
```

In [299]:

```
rf_model.fit(X_train, y_train)
```

Out[299]:



In [300]:

```
y_pred_rf = rf_model.predict(X_test)
```

In [301]:

```
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]
```

In [302]:

```
print(y_pred_rf[:10])
print(y_prob_rf[:10])
```

```
[0 1 0 0 0 0 0 0 0 0]
[0.  0.74 0.04 0.35 0.01 0.34 0.36 0.05 0.01 0.43]
```

In [303]:

```
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)
roc_auc_rf = roc_auc_score(y_test, y_prob_rf)
```

```
print("Precision:", precision_rf)
print("Recall:", recall_rf)
print("F1 Score:", f1_rf)
print("ROC-AUC:", roc_auc_rf)
```

```
Precision: 0.6118881118881119
Recall: 0.4679144385026738
F1 Score: 0.5303030303030303
ROC-AUC: 0.8222299723578497
```

Random Forest Results

The Random Forest achieved a precision of 0.612 recall of 0.468 F1 score of 0.530 and ROC AUC of 0.822. The model achieved higher precision than the previous models but its recall was lower than Logistic

Regression. This means it was more precise when predicting churn but missed more actual churn customers.

super vectore machine

In [304]:

```
from sklearn.svm import SVC
```

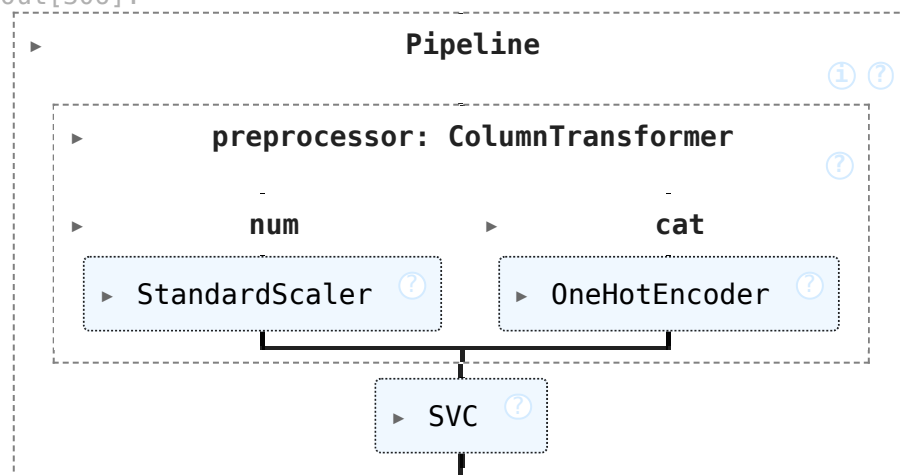
In [305]:

```
svm_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", SVC(
        class_weight="balanced",
        probability=True,
        random_state=42
    ))
])
```

In [306]:

```
svm_model.fit(X_train, y_train)
```

Out[306]:



In [307]:

```
y_pred_svm = svm_model.predict(X_test)
```

In [308]:

```
y_prob_svm = svm_model.predict_proba(X_test)[:, 1]
```

In [309]:

```
print(y_pred_svm[:10])
print(y_prob_svm[:10])
```

```
[0 1 0 1 0 1 1 0 0 1]
[0.05691009 0.69047815 0.05378376 0.34643634 0.05824535 0.67210216
 0.48645152 0.08178063 0.07576371 0.37459648]
```

In [310]:

```
precision_svm = precision_score(y_test, y_pred_svm)
recall_svm = recall_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm)
roc_auc_svm = roc_auc_score(y_test, y_prob_svm)
```

```

print("Precision:", precision_svm)
print("Recall:", recall_svm)
print("F1 Score:", f1_svm)
print("ROC-AUC:", roc_auc_svm)

```

```

Precision: 0.5364485981308411
Recall: 0.767379679144385
F1 Score: 0.6314631463146315
ROC-AUC: 0.8245730966958588

```

Model Comparison Four classification models were trained and evaluated using Precision Recall F1 score and ROC AUC. SVM achieved the highest F1 score of 0.631 while Logistic Regression achieved the highest ROC AUC of 0.847 and the highest recall of 0.799. Since customer churn detection is important for retention recall is an important business metric because missing a customer who is likely to churn can result in a lost customer.

GridSearchCV

In [311]:

```

from sklearn.model_selection import GridSearchCV

```

In [312]:

```

param_grid = {
    "classifier__n_estimators": [100, 200],
    "classifier__max_depth": [None, 10, 20]
}

```

In [313]:

```

grid_search = GridSearchCV(
    estimator=rf_model,
    param_grid=param_grid,
    scoring="f1",
    cv=5,
    n_jobs=-1
)

```

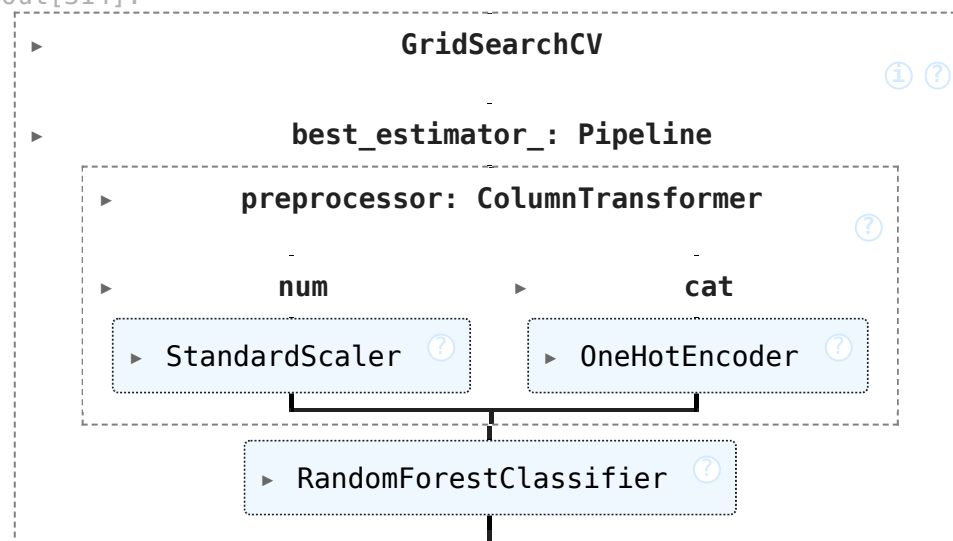
In [314]:

```

grid_search.fit(X_train, y_train)

```

Out[314]:



In [315]:

```
print("Best Parameters:", grid_search.best_params_)
```

Best Parameters: {'classifier__max_depth': 10, 'classifier__n_estimators': 200}

In [316]:

```
print("Best CV F1 Score:", grid_search.best_score_)
```

Best CV F1 Score: 0.6351680800402499

In [317]:

```
best_rf = grid_search.best_estimator_  
  
y_pred_rf_tuned = best_rf.predict(X_test)  
y_prob_rf_tuned = best_rf.predict_proba(X_test)[:, 1]  
  
print("First 10 Predictions:", y_pred_rf_tuned[:10])  
print("First 10 Probabilities:", y_prob_rf_tuned[:10])
```

First 10 Predictions: [0 1 0 0 0 1 1 0 0 1]

First 10 Probabilities: [0.01333828 0.80319548 0.11344098 0.46715022 0.04004848 0.75713123

0.51614087 0.11441037 0.0096331 0.5661335]

In [318]:

```
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score  
  
precision_rf_tuned = precision_score(y_test, y_pred_rf_tuned)  
recall_rf_tuned = recall_score(y_test, y_pred_rf_tuned)  
f1_rf_tuned = f1_score(y_test, y_pred_rf_tuned)  
roc_auc_rf_tuned = roc_auc_score(y_test, y_prob_rf_tuned)  
  
print("Tuned Random Forest")  
print("Precision:", precision_rf_tuned)  
print("Recall:", recall_rf_tuned)  
print("F1-score:", f1_rf_tuned)  
print("ROC-AUC:", roc_auc_rf_tuned)
```

Tuned Random Forest

Precision: 0.5541666666666667

Recall: 0.7112299465240641

F1-score: 0.6229508196721312

ROC-AUC: 0.8405177090599086

Tuned Random Forest Evaluation

GridSearchCV was used to tune the Random Forest model using `n_estimators` and `max_depth`. The best parameters were 200 trees and a maximum depth of 10. The tuned model achieved an F1 score of 0.623 and ROC AUC of 0.841 on the test set. Compared with the default Random Forest tuning substantially improved recall from 0.468 to 0.711 making the model better at identifying customers who are likely to churn.

In [319]:

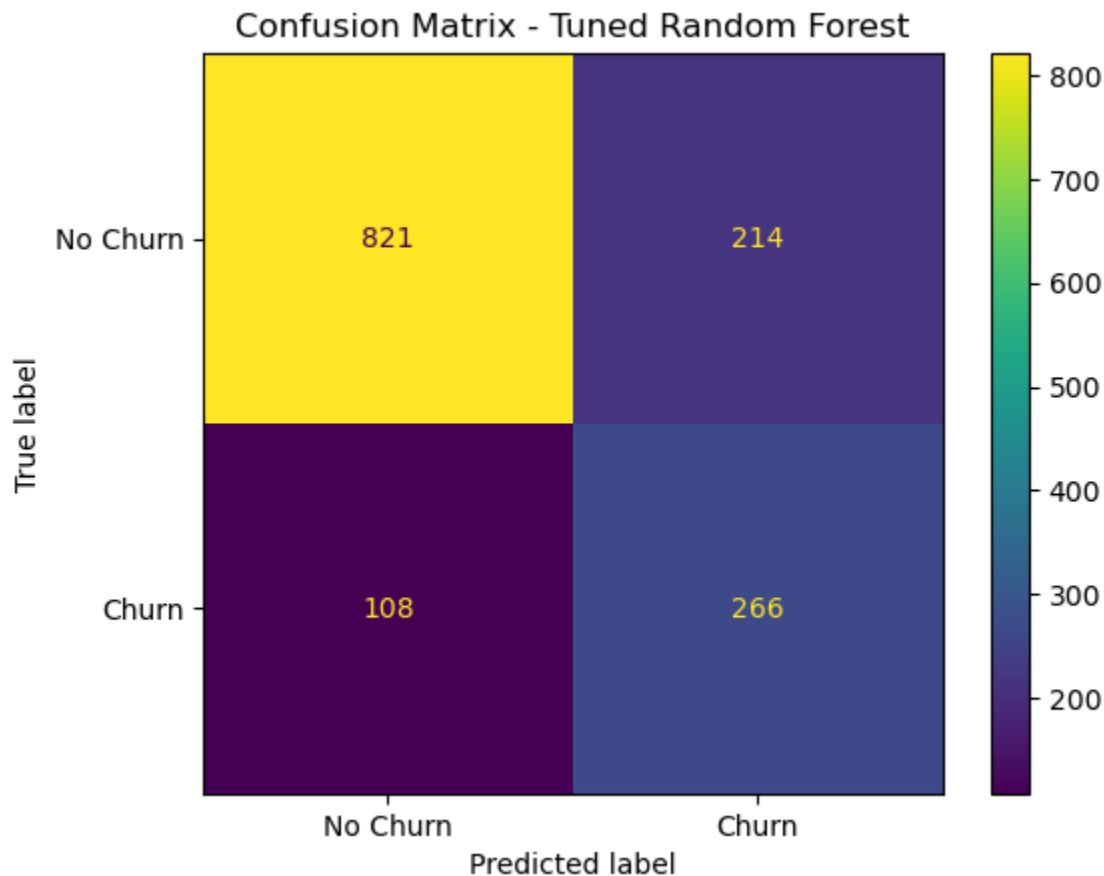
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay  
import matplotlib.pyplot as plt  
  
cm = confusion_matrix(y_test, y_pred_rf_tuned)
```

```

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Churn", "Churn"]
)

disp.plot()
plt.title("Confusion Matrix - Tuned Random Forest")
plt.show()

```



Confusion Matrix Analysis

The confusion matrix of the tuned Random Forest shows 821 true negatives 214 false positives 108 false negatives and 266 true positives. The model correctly identified 266 customers who churned. However 108 actual churners were incorrectly classified as non churners. Since customer retention is the main business goal reducing false negatives is important because missed churners may result in lost customers.

In [320]:

```

from sklearn.metrics import precision_score, recall_score, f1_score

threshold = 0.4

y_pred_threshold = (y_prob_rf_tuned >= threshold).astype(int)

precision_threshold = precision_score(y_test, y_pred_threshold)
recall_threshold = recall_score(y_test, y_pred_threshold)
f1_threshold = f1_score(y_test, y_pred_threshold)

print("Threshold:", threshold)
print("Precision:", precision_threshold)

```

```
print("Recall:", recall_threshold)
print("F1-score:", f1_threshold)
```

Threshold: 0.4
Precision: 0.5190972222222222
Recall: 0.7994652406417112
F1-score: 0.6294736842105263

In [321]:

```
preprocessor_fitted = best_rf.named_steps["preprocessor"]
rf_classifier = best_rf.named_steps["classifier"]

feature_names = preprocessor_fitted.get_feature_names_out()

importances = rf_classifier.feature_importances_

feature_importance = pd.Series(
    importances,
    index=feature_names
).sort_values(ascending=False)

print(feature_importance.head(10))
```

cat__Contract_Month-to-month	0.113915
num__tenure	0.101088
num__TotalCharges	0.094140
num__MonthlyCharges	0.069484
num__AvgMonthlyCharge	0.067798
cat__TechSupport_No	0.051728
cat__Contract_Two year	0.046222
cat__InternetService_Fiber optic	0.044976
cat__OnlineSecurity_No	0.043178
num__IsNewCustomer	0.034422

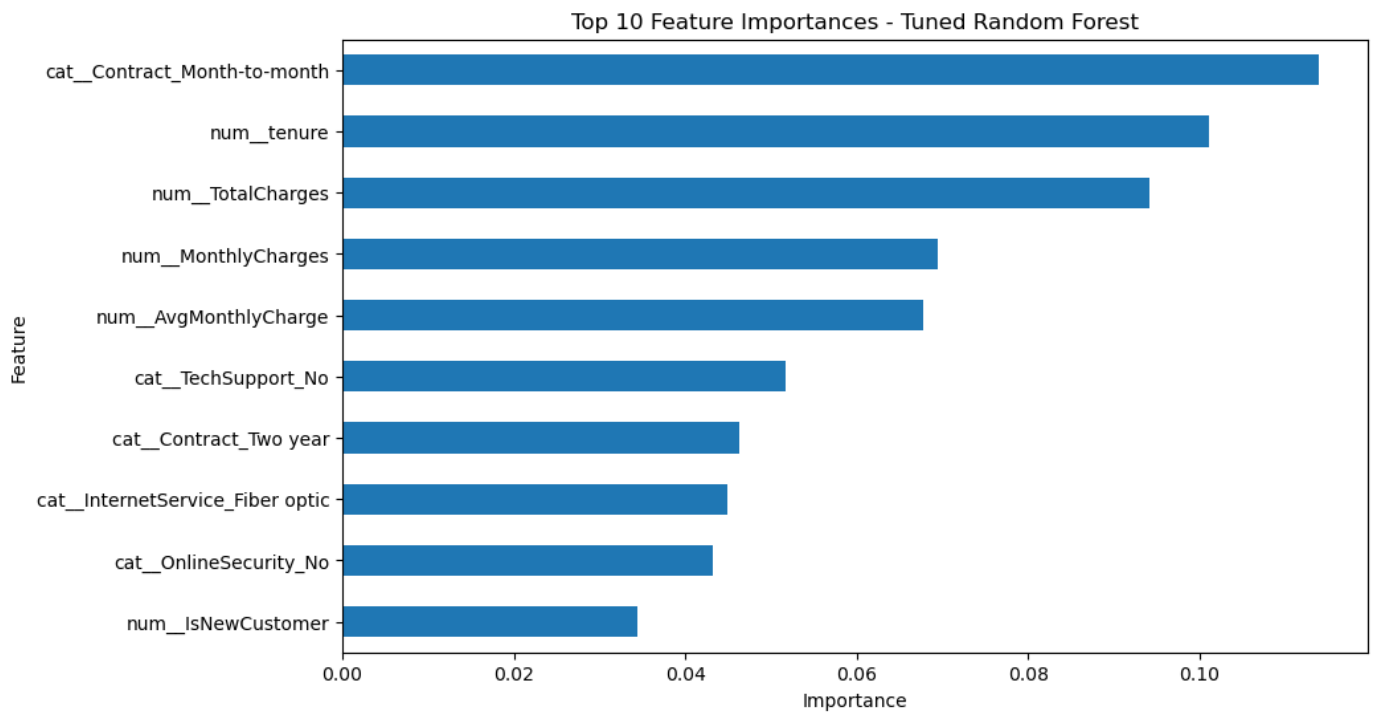
dtype: float64

In [322]:

```
top_10 = feature_importance.head(10).sort_values()

plt.figure(figsize=(10, 6))
top_10.plot(kind="barh")

plt.title("Top 10 Feature Importances - Tuned Random Forest")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()
```



In [323]:

```
results = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Decision Tree",
        "Random Forest",
        "SVM",
        "Tuned Random Forest"
    ],
    "Precision": [
        0.5068,
        0.4947,
        0.6119,
        0.5364,
        precision_rf_tuned
    ],
    "Recall": [
        0.7995,
        0.4973,
        0.4679,
        0.7674,
        recall_rf_tuned
    ],
    "F1-score": [
        0.6203,
        0.4960,
        0.5303,
        0.6315,
        f1_threshold
    ],
    "ROC-AUC": [
        0.8472,
        0.6566,
        0.8222,
        0.8246,
        roc_auc_rf_tuned
    ]
})
```

```
})
```

```
results
```

```
Out[323]:
```

	Model	Precision	Recall	F1-score	ROC-AUC
0	Logistic Regression	0.506800	0.79950	0.620300	0.847200
1	Decision Tree	0.494700	0.49730	0.496000	0.656600
2	Random Forest	0.611900	0.46790	0.530300	0.822200
3	SVM	0.536400	0.76740	0.631500	0.824600
4	Tuned Random Forest	0.554167	0.71123	0.629474	0.840518

Final Model Selection

Based on the model comparison Logistic Regression was selected as the final model for the customer churn prediction task. It achieved the highest recall of 0.7995 and the highest ROC AUC of 0.8472 among the evaluated models. Since the main business objective is customer retention recall is particularly important because failing to identify a customer who is likely to churn can result in customer loss. Although SVM achieved a slightly higher F1 score of 0.6315 Logistic Regression provided better recall and ROC AUC making it more suitable for this business problem.

Executive Summary

The objective of this project was to predict customer churn for a telecommunications company and identify customers who are at risk of leaving. The Telco Customer Churn dataset was cleaned explored and prepared using appropriate preprocessing techniques. Three additional features were engineered: AvgMonthlyCharge IsNewCustomer and ServiceCount.

Four classification models were evaluated: Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine (SVM). A tuned Random Forest was also developed using GridSearchCV by optimizing the number of trees and maximum tree depth. Among the evaluated models Logistic Regression achieved the highest recall of 0.7995 and the highest ROC AUC of 0.8472. Therefore, Logistic Regression was selected as the final model because identifying potential churners is particularly important for customer retention.

Feature importance from the tuned Random Forest showed that Contract type (especially month-to-month contracts), tenure, and TotalCharges were among the most important features. Other important factors included MonthlyCharges AvgMonthlyCharge lack of TechSupport Fiber optic InternetService lack of OnlineSecurity and being a new customer.

The analysis suggests that the company should pay particular attention to customers on month to month contracts and customers with shorter tenure. The company could offer targeted retention incentives, encourage longer term contracts, and provide additional support services to customers at higher risk of churn. Lowering the prediction threshold from 0.5 to 0.4 increased recall to 0.7995, helping identify more potential churners although precision decreased. Overall the developed models can support the company in identifying high risk customers and taking proactive retention actions.

In []: